哈尔滨工业大学（深圳）

软件构造
实验指导书

实验四 DAO 模式和单元测试

2026 春

目录

1. 实验目的.....3
2. 实验环境.....3
3. 实验内容.....3
4. 实验步骤.....3
4.1 数据访问对象（DAO）模式应用.....3
4.1.1 排行榜数据持久化场景分析.....3
4.1.2 DAO 模式代码实现.....4
4.2 单元测试.....8
4.2.1 用 JUnit5 进行单元测试.....8
4.2.2 JUnit5 的常见用法.....13
（1）JUnit5 注解（Annotations）.....13
（2）JUnit5 断言（Assertions）.....15
（3）JUnit5 假设（Assumptions）.....17
（4）JUnit5 测试异常（Test Exception）.....19
（5）JUnit5 参数测试（Parameterized Tests ）.....20
5. 本次迭代开发目标.....20

1. 实验目的

1. 深入理解数据访问对象（DAO）模式的设计意图及结构
2. 结合案例，规范绘制 DAO 模式的 UML 结构图
3. 掌握重构方法，通过代码实现并应用 DAO 模式
4. 了解单元测试的定义及其重要性
5. 掌握使用 JUnit5 进行单元测试的常用方法

2. 实验环境

1. IntelliJ IDEA 2025.3
2. OpenJDK 2
3. JUnit5

3. 实验内容

1. 深入理解数据访问对象（DAO）模式的设计原理与结构，并结合实例规范绘制该模式的 UML 结构图；
2. 完成代码重构，采用 DAO 模式实现玩家得分排行榜的相关功能（数据访问、存储与展示）；
3. 对英雄机类进行单元测试，至少选择 3 个方法（含其父类方法）作为测试对象，设计合理测试用例并完成测试；
4. 完成本次迭代开发清单中的全部任务。

4. 实验步骤

4.1 数据访问对象（DAO）模式应用

4.1.1 排行榜数据持久化场景分析

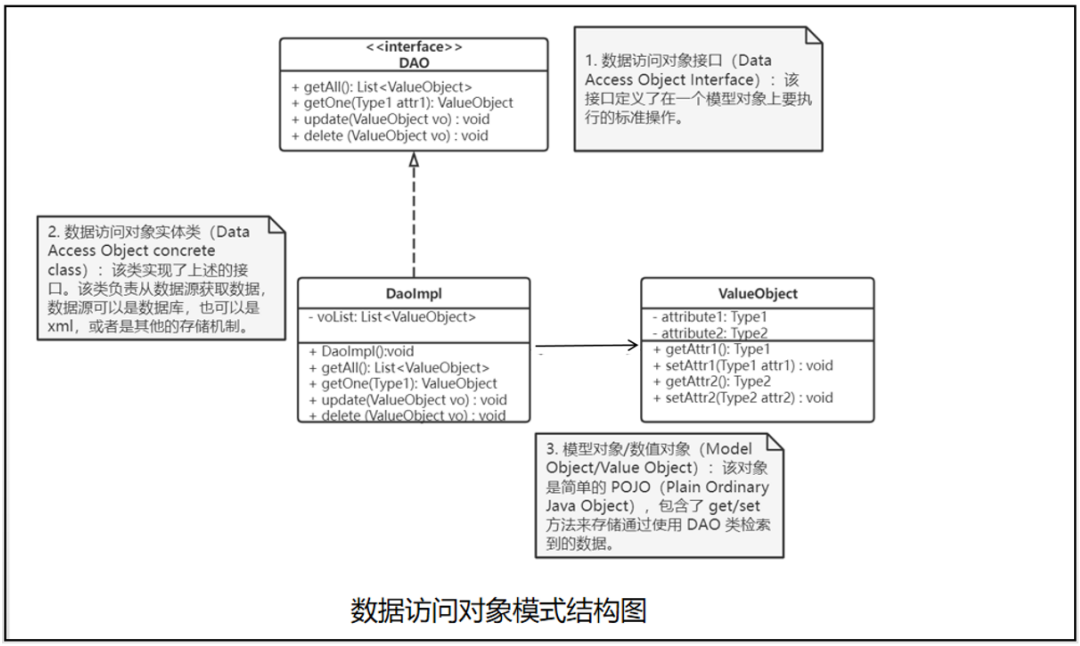
在飞机大战游戏中，系统实时记录英雄机当前得分并在界面左上角实时显示，英雄机每击落一架敌机即可获得对应分数。每局游戏结束后，将展示当前难度下的总分排行榜，榜单内容包含名次、玩家名、得分及记录时间。游戏结束时系统会询问玩家是否保存本局记录，若确认保存，则将该条游戏成绩插入到数据记录中，玩家也可选择删除指定的游戏记录。

设计任务：鉴于游戏需要对玩家得分记录进行新增、查询、排序展示等数据操作，且业务逻辑与数据访问逻辑需分离以提高可维护性与可扩展性，请采用数据访问对象（DAO）模式进行排行榜数据管理类的设计与代码重构。

UML 建模：绘制该模式的 UML 类图，图中需明确标注设计模式名称，并自定义合理的类名、属性名及方法名，以完整展示 DAO 模式的实现结构。

数据访问对象模式

数据访问对象模式（Data Access Object Pattern）或 DAO 模式用于把低级的数据访问 API 或操作从高级的业务服务中分离出来。



数据访问对象模式结构图

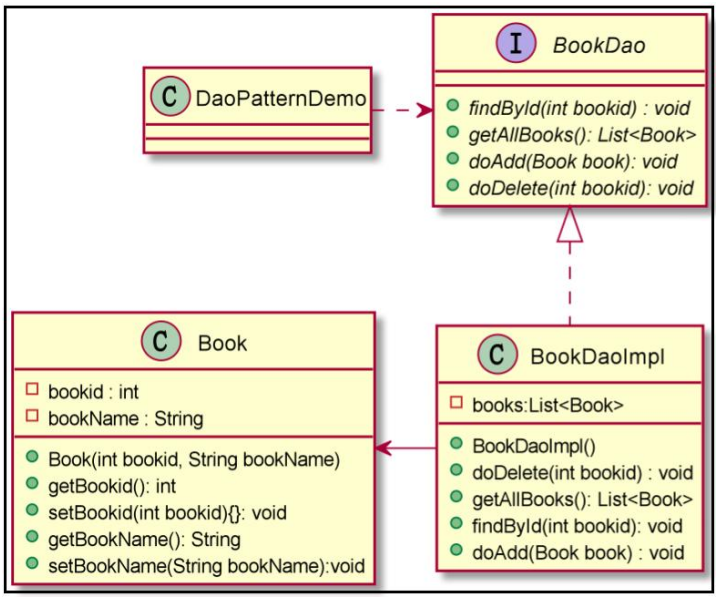
请参考以上 UML 结构图，绘制飞机大战中的数据访问对象模式。

4.1.2 DAO 模式代码实现

数据访问对象模式代码示例：

假如我们要实现一个图书管理系统，需完成查询所有图书、按编号查询图书、新增图书、删除图书的功能，我们该如何绘制其 UML 类图？

我们将创建一个作为数值对象的 Book 类（封装图书信息）；BookDao 作为数据访问对象接口，定义所有数据访问相关方法；BookDaoImpl 作为实现该接口的实体类，具体实现接口中的所有数据操作逻辑；DaoPatternDemo 作为演示类，将通过调用 BookDao 接口及其实现类，演示数据访问对象（DAO）模式的具体用法。下图是将 DAO 模式应用于该图书管理问题的 UML 类图设计。



步骤 1：创建数值对象 VO 实体类。

Book.java

```
public class Book {
    private int bookid;
    private String bookName;

    Book(int bookid, String bookName) {
        this.bookid = bookid;
        this.bookName = bookName;
    }

    public int getBookid() {
        return bookid;
    }

    public void setBookid(int bookid) {
        this.bookid = bookid;
    }

    public String getBookName() {
        return bookName;
    }

    public void setBookName(String bookName) {
        this.bookName = bookName;
    }

}
```

步骤 2：创建数据访问对象接口。

BookDao.java

```
public interface BookDao {
    void findById(int bookid);

    List<Book> getAllBooks();

    void doAdd(Book book);

    void doDelete(int bookid);
}
```

步骤 3：创建实现了上述接口的 DAO 实现类。

BookDaoImpl.java

```
public class BookDaoImpl implements BookDao {

    //模拟数据库数据
    private List<Book> books;

    public BookDaoImpl() {
        books = new ArrayList<Book>();
        books.add(new Book(1001, "Clean Code"));
        books.add(new Book(1002, "Design Patterns"));
        books.add(new Book(1003, "Effective Java"));
    }

    //删除图书
    @Override
    public void doDelete(int bookid) {
        for (Book item : books) {
            if (item.getBookid() == bookid) {
                books.remove(item);
                System.out.println("Delete Book: ID [" + bookid + "]");
                return;
            }
        }
        System.out.println("Can not find this book!");
    }

    //获取所有图书
    @Override
    public List<Book> getAllBooks() {
        return books;
    }

    //查找图书
    @Override
    public void findById(int bookid) {
        for (Book item : books) {
```

```
                if (item.getBookid() == bookid) {
                        System.out.println("Find Book: ID [" + bookid + "],
                                                Book Name [" + item.getBookName() + "]");
                        return;
                }
        }
        System.out.println("Can not find this book!");
    }

    //新增图书
    @Override
    public void doAdd(Book book) {
        books.add(book);
        System.out.println("Add new Book: ID [" + book.getBookid() + "],
                                Book Name [" + book.getBookName() + "]");
    }
}
```

步骤 4：使用 DaoPatternDemo 来演示数据访问对象模式的用法。

DaoPatternDemo.java

```
public class DaoPatternDemo {
    public static void main(String[] args) {

        BookDao bookDao = new BookDaoImpl();

        //输出所有图书
        for (Book book : bookDao.getAllBooks()) {
            System.out.println("Book ID [" + book.getBookid() + "],
                                    Book Name : [" + book.getBookName() + "]");
        }
        System.out.println("----------------------------------------------");

        //查找图书
        bookDao.findById(1002);
        System.out.println("----------------------------------------------");

        //删除图书
        bookDao.doDelete(1002);
        System.out.println("----------------------------------------------");

        //新增图书
        Book newBook = new Book(1004, "Thinking In java");
        bookDao.doAdd(newBook);
        System.out.println("----------------------------------------------");

        //输出所有图书
        for (Book book : bookDao.getAllBooks()) {
            System.out.println("Book ID [" + book.getBookid() + "],
                                    Book Name : [" + book.getBookName() + "]");
        }
    }
}
```

步骤 5：执行程序，输出结果：

```
Book ID [1001], Book Name : [Clean Code]
Book ID [1002], Book Name : [Design Patterns]
Book ID [1003], Book Name : [Effective Java]
--------------------------------------------------------------------------
Find Book: ID [1002], Book Name [Design Patterns]
--------------------------------------------------------------------------
Delete Book: ID [1002]
--------------------------------------------------------------------------
Add new Book: ID [1004], Book Name [Thinking In java]
--------------------------------------------------------------------------
Book ID [1001], Book Name : [Clean Code]
Book ID [1003], Book Name : [Effective Java]
Book ID [1004], Book Name : [Thinking In java]
```

4.2 单元测试

单元测试（Unit Testing），是指对软件中的最小可测试单元进行检查和验证。对于面向对象编程，最小单元就是方法，包括基类（超类）、抽象类、或者派生类（子类）中的方法。

JUnit 是什么？

JUnit 是一个 Java 编程语言的单元测试框架。它是由 Kent Beck （极限编程）和 Erich Gamma（设计模式）建立，是 xUnit 家族中最成功的的一个。大部分的 Java IDE 都集成了 JUnit 作为单元测试工具。JUnit 在测试驱动的开发方面有很重要的发展。

官方文档：JUnit User Guide

4.2.1 用 JUnit5 进行单元测试

在 IntelliJ IDEA 中创建一个 Java Project （本例中 project 名为 JunitDemo），在 package junit.demo 下创建类 Calculator.java,在其中编写 5 个方法，用于将参数 x,y 的值进行加、减、乘、除并且返回结果。

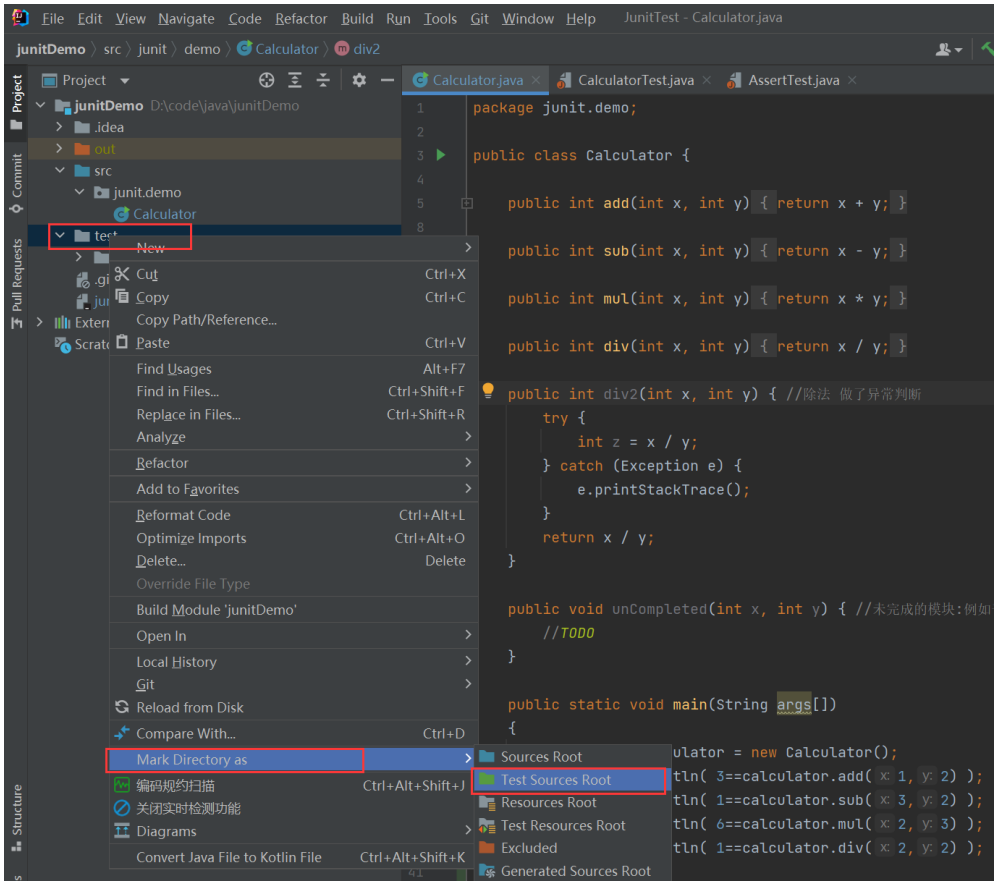
Calculator.java

```
package junit.demo;
public class Calculator {
    public int add(int x, int y) { //加法
        return x + y;
    }

    public int sub(int x, int y) { //减法
        return x - y;
    }
```

```
    public int mul(int x, int y) { //乘法
        return x * y;
    }

    public int div(int x, int y) { //除法
        return x / y;
    }

    public int div2(int x, int y) { //除法  做了异常判断
        try {
            int z = x / y;
        } catch (Exception e) {
            e.printStackTrace();
        }
        return x / y;
    }

    public void unCompleted(int x, int y) { //未完成的模块:例如平方、开方等等
        //TODO
    }
}
```

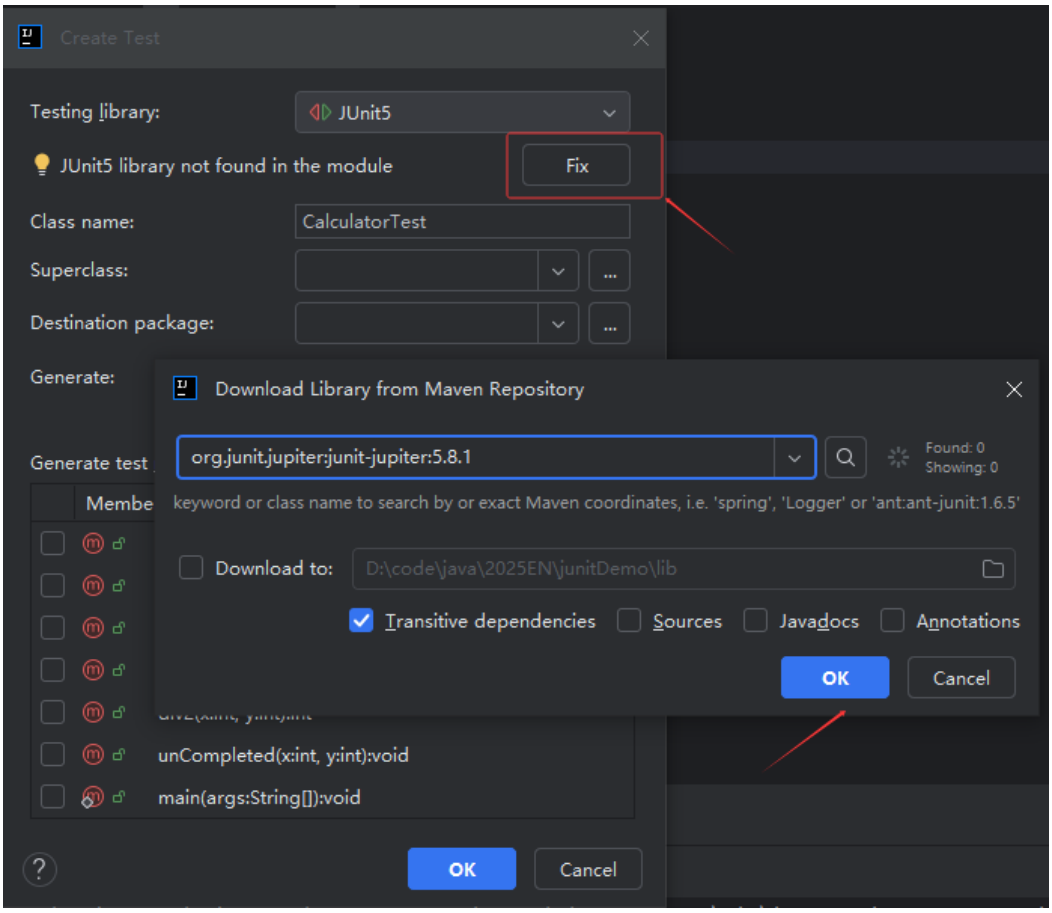
（1）新建一个 test 的文件夹，用于存放单元测试类，然后标记为测试类文件夹。

右键点击 test 文件夹，选择 Mark Directory as -> Test Sources Root

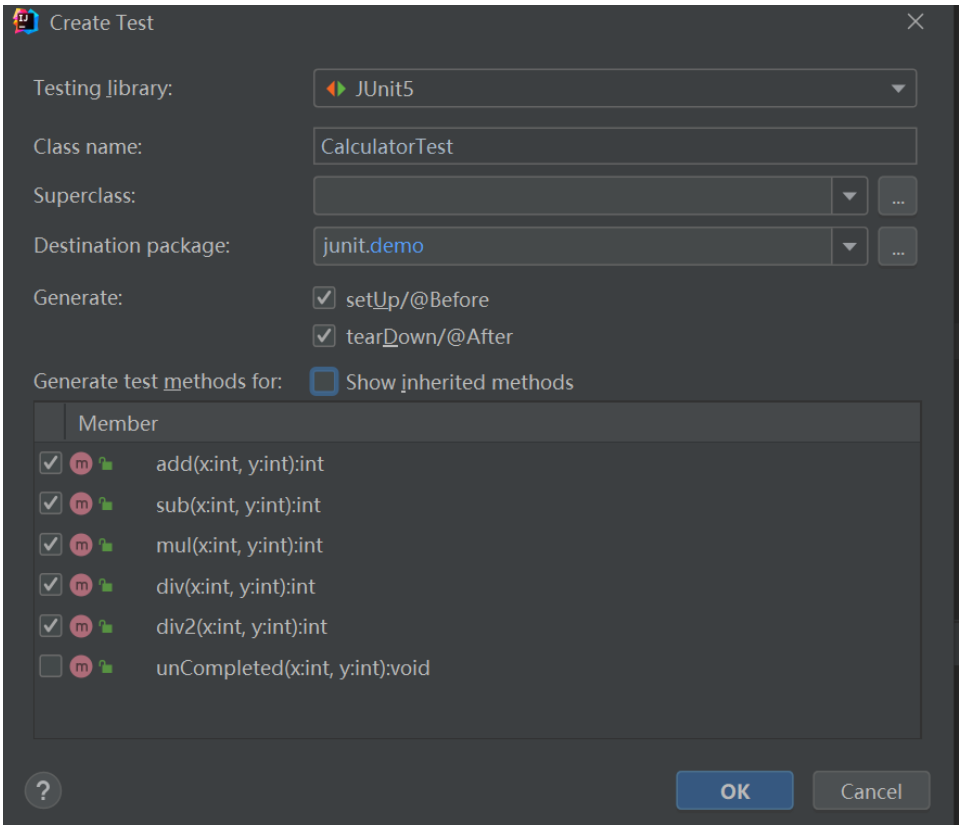


（2）打开 Calculator . java 文件，在类的内部同时按下快捷键 ctrl + shift + T，选择 Create New Test（或右键选择 Generate -> Test）。

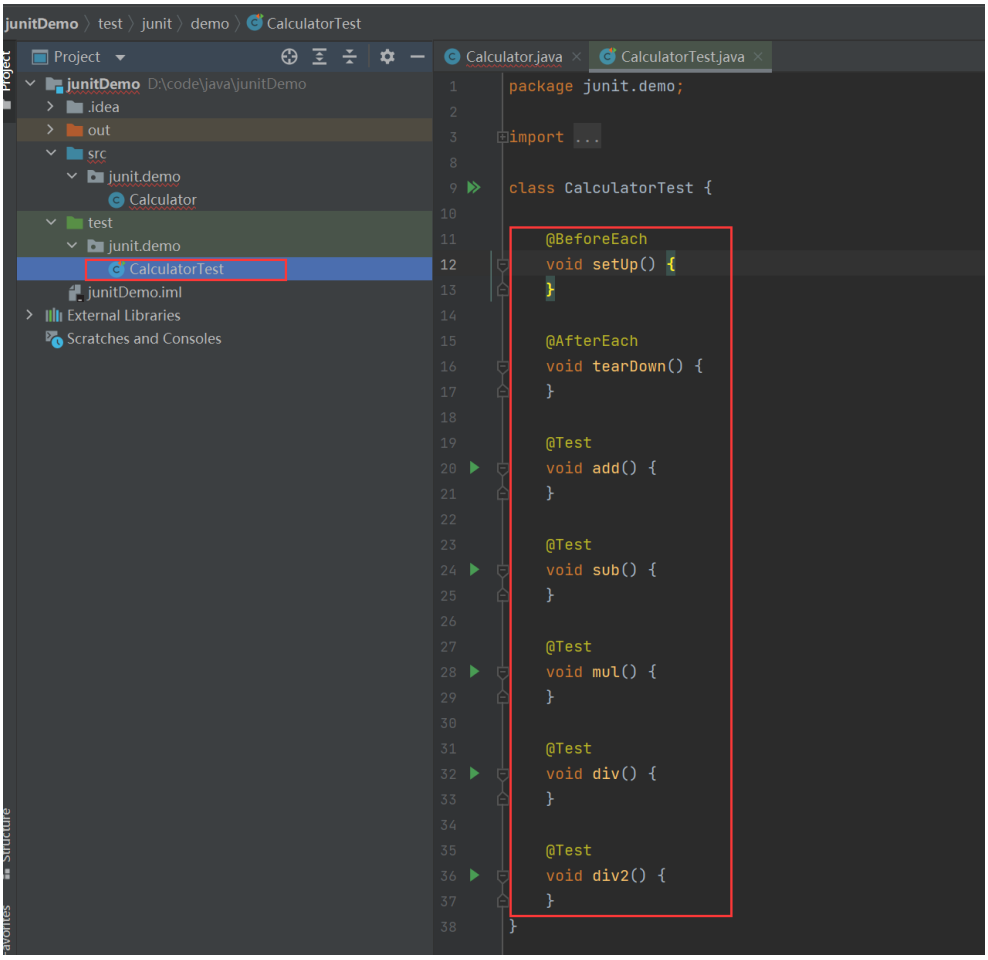
若之前没有安装过 JUnit5 Library， 在界面中点击 Fix 按钮，IDEA 会帮助你下载好 JUnit5（点击 OK）。



（3）安装后在 Member 中勾选 Calculator 中需要进行单元测试的方法。



（4）查看 test 目录下自动生成的单元测试类 CalculatorTest。



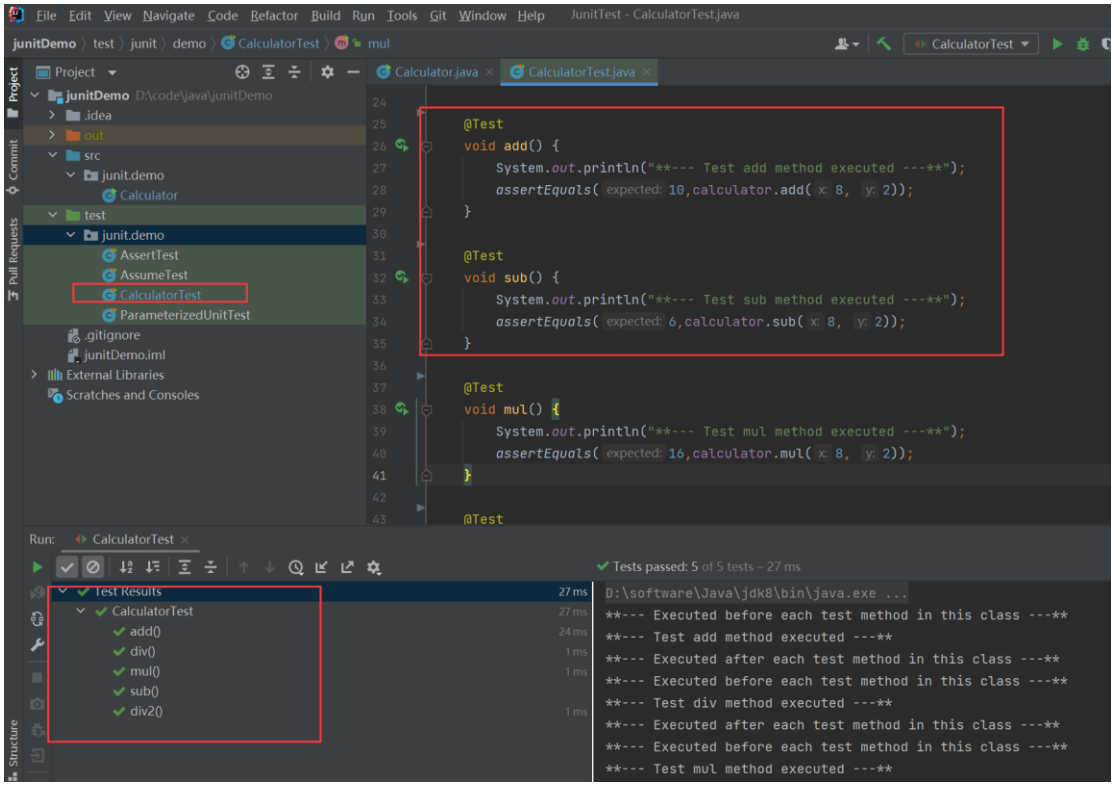
（5）采用黑盒测试和白盒测试的常用方法，设计测试用例，并编写单元测试代码，代码修改详见 4.2.2 节。

CalculatorTest.java

```
package junit.demo;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {

	private Calculator calculator;

	@BeforeEach
	void setUp() {
		System.out.println("**--- Executed before each test method in this class ---**");
		calculator = new Calculator();
	}

	@AfterEach
	void tearDown() {
		System.out.println("**--- Executed after each test method in this class ---**");
		calculator = null;
	}

	@Test
	void add() {
		System.out.println("**--- Test add method executed ---**");
		assertEquals(10,calculator.add(8, 2));
	}

	@Test
	void sub() {
		System.out.println("**--- Test sub method executed ---**");
		assertEquals(6,calculator.sub(8, 2));
	}

	@Test
	void mul() {
		System.out.println("**--- Test mul method executed ---**");
		assertEquals(16,calculator.mul(8, 2));
	}

	@Test
	void div() {
		System.out.println("**--- Test div method executed ---**");
		assertEquals(4,calculator.div(8, 2));
	}

	//Todo
```

```
    @Test
    void div2() {
    }
}
```

（6）右键 CalculatorTest 类，选择 Run CalculatorTest， 即可运行该单元测试类。



4.2.2 JUnit5 的常见用法

（1）JUnit5 注解（Annotations）

下面列出了 JUnit5 提供的一些常用注解：

Annotation	Description
@Test	Denotes a test method
@DisplayName	Declares a custom display name for the test class or test method
@BeforeEach	Denotes that the annotated method should be executed before each test method
@AfterEach	Denotes that the annotated method should be executed after each test method
@BeforeAll	Denotes that the annotated method should be executed before all test methods
@AfterAll	Denotes that the annotated method should be executed after all test methods
@Disable	Used to disable a test class or test method
@Nested	Denotes that the annotated class is a nested, non-static test class
@Tag	Declare tags for filtering tests
@ExtendWith	Register custom extensions

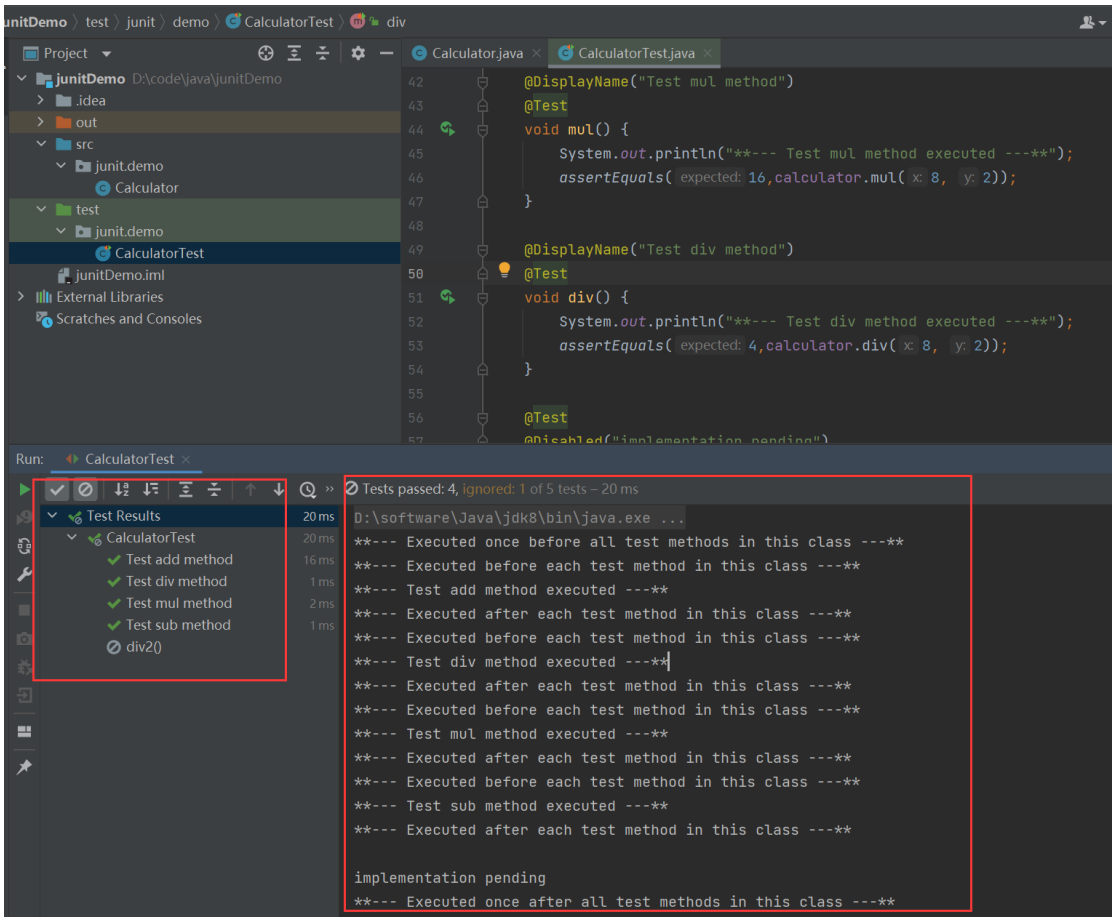
代码示例：

```
package junit.demo;

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {

	private Calculator calculator;

	@BeforeAll
	static void beforeAll() {
		System.out.println("**--- Executed once before all test methods in this class ---**");
	}

	@BeforeEach
	void setUp() {
		System.out.println("**--- Executed before each test method in this class ---**");
		calculator = new Calculator();
	}

	@AfterEach
	void tearDown() {
		System.out.println("**--- Executed after each test method in this class ---**");
		calculator = null;
	}

	@DisplayName("Test add method")
	@Test
	void add() {
		System.out.println("**--- Test add method executed ---**");
		assertEquals(10,calculator.add(8, 2));
	}

	@DisplayName("Test sub method")
	@Test
	void sub() {
		System.out.println("**--- Test sub method executed ---**");
		assertEquals(6,calculator.sub(8, 2));
	}

	@DisplayName("Test mul method")
	@Test
	void mul() {
		System.out.println("**--- Test mul method executed ---**");
		assertEquals(16,calculator.mul(8, 2));
	}

	@DisplayName("Test div method")
	@Test
	void div() {
		System.out.println("**--- Test div method executed ---**");
		assertEquals(4,calculator.div(8, 2));
	}
```

```
    @Test
    @Disabled("implementation pending")
    void div2() {
    }

    @AfterAll
    static void afterAll() {
        System.out.println("**--- Executed once after all test methods in this class ---**");
    }

}
```

运行结果：

从测试结果中我们可以看到 div2 用例被禁用，没有被执行。



（2）JUnit5 断言（Assertions）

必须使用断言将每个测试方法的条件评估为 true，以便测试可以继续执行。JUnit Jupiter 断言保存在 org.junit.jupiter.api.Assertions 类中，所有方法都是静态的。

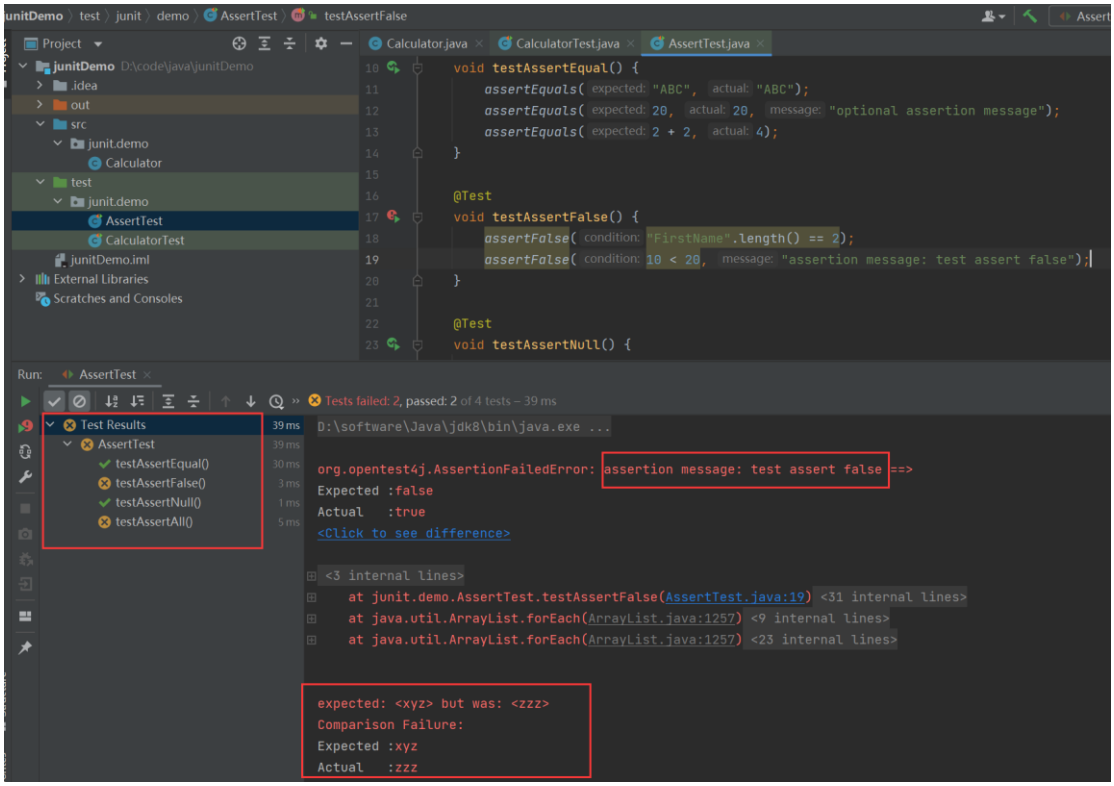
Assertion	Description
assertEquals(expected, actual)	Fails when expected does not equal actual
assertFalse(expression)	Fails when expression is not false
assertNull(actual)	Fails when actual is not null
assertNotNull(actual)	Fails when actual is null
assertAll()	Group many assertions and every assertion is executed even if one or more of them fails
assertTrue(expression)	Fails if expression is not true
assertThrows()	Class to be tested is expected to throw an exception

代码示例：

```
package junit.demo;

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

public class AssertTest {

	@Test
	void testAssertEqual() {
		assertEquals("ABC", "ABC");
		assertEquals(20, 20, "optional assertion message");
		assertEquals(2 + 2, 4);
	}

	@Test
	void testAssertFalse() {
		assertFalse("FirstName".length() == 2);
		assertFalse(10 < 20, "assertion message: test assert false");
	}

	@Test
	void testAssertNull() {
		String str1 = null;
		String str2 = "abc";
		assertNull(str1);
		assertNotNull(str2);
	}

	@Test
	void testAssertAll() {
		String str1 = "abc";
		String str2 = "pqr";
		String str3 = "xyz";
		assertAll(
			() -> assertEquals(str1,"abc"),
			() -> assertEquals(str2,"pqr"),
			() -> assertEquals(str3,"zzz")
		);
```

```
    }
}
```

运行结果：



（3）JUnit5 假设（Assumptions）

假设是 org.junit.jupiter.api.Assumptions 类中的静态方法。 他们仅在满足指定条件时执行测试，否则测试将中止。 中止的测试不会导致构建失败。 当假设失败时，将抛出 org.opentest4j.TestAbortedException 并跳过测试。

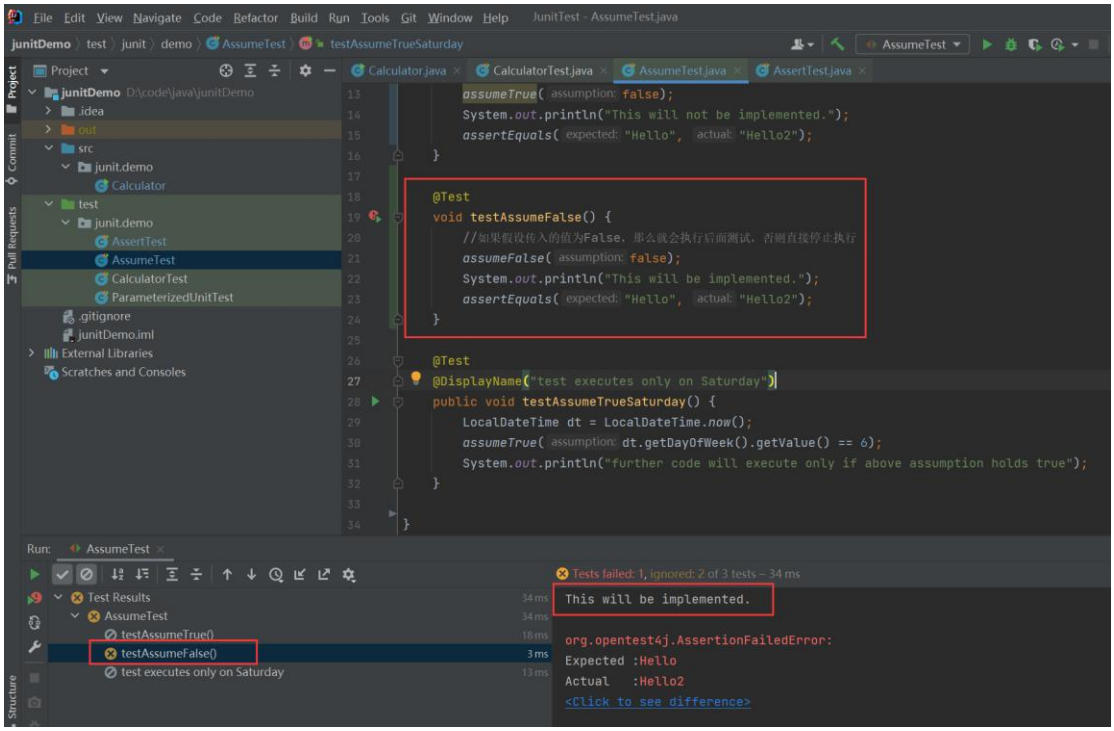
Assumptions	Description
assumeTrue	Execute the body of lamda when the positive condition hold else test will be skipped
assumeFalse	Execute the body of lamda when the negative condition hold else test will be skipped
assumingThat	Portion of the test method will execute if an assumption holds true and everything after the lambda will execute irrespective of the assumption in assumingThat() holds

代码示例：

```
package junit.demo;

import org.junit.jupiter.api.*;
import java.time.LocalDateTime;
import static org.junit.jupiter.api.Assertions.*;
import static    org.junit.jupiter.api.Assumptions.*;
```

```
public class AssumeTest {

    @Test
    void testAssumeTrue() {
        //如果假设传入的值为True，那么就会执行后面测试，否则直接停止执行
        assumeTrue(false);
        System.out.println("This will not be implemented.");
        assertEquals("Hello", "Hello2");
    }

    @Test
    void testAssumeFalse() {
        //如果假设传入的值为False，那么就会执行后面测试，否则直接停止执行
        assumeFalse(false);
        System.out.println("This will be implemented.");
        assertEquals("Hello", "Hello2");
    }

    @Test
    @DisplayName("test executes only on Saturday")
    public void testAssumeTrueSaturday() {
        LocalDateTime dt = LocalDateTime.now();
        assumeTrue(dt.getDayOfWeek().getValue() == 6);
        System.out.println("further code will execute only if above assumption holds true");
    }


}
```

运行结果：



（4）JUnit5 测试异常（Test Exception）

在某些情况下，期望方法在特定条件下引发异常。 如果给定方法未引发指定的异常，则 assertThrows 将使测试失败。

JUnit5 assertThrows()的语法：
它断言所提供的 executable 的执行将引发 expectedType 的异常并返回该异常。
public static <T extends Throwable> T assertThrows(Class<T> expectedType, Executable executable)

代码示例：

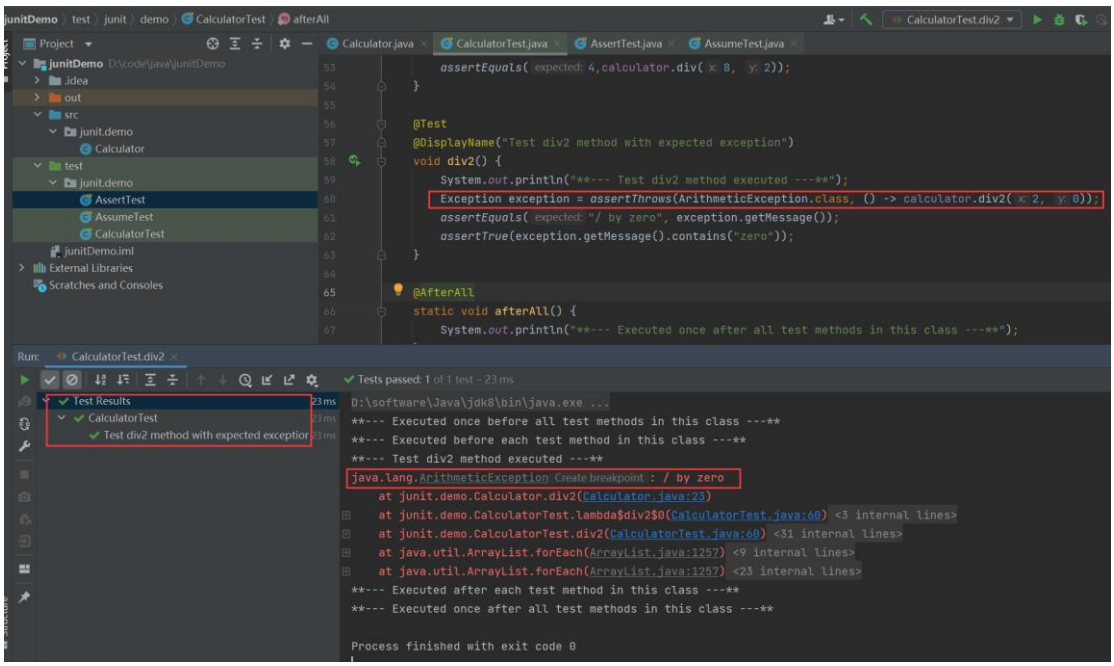
被测方法：

```
public int div2(int x, int y) { //除法 做了异常判断
        try {
            int z = x / y;
        } catch (Exception e) {
            e.printStackTrace();
        }
        return x / y;
}
```

测试方法：

```
@Test
@DisplayName("Test div2 method with expected exception")
void div2() {
    System.out.println("**--- Test div2 method executed ---**");
    Exception exception = assertThrows(ArithmeticException.class, () -> calculator.div2(2, 0));
    assertEquals("/ by zero", exception.getMessage());
    assertTrue(exception.getMessage().contains("zero"));
}
```

运行结果：



（5）JUnit5 参数测试（Parameterized Tests ）

要使用 JUnit 5 进行参数化测试，除了 junit-jupiter-engine 基础依赖之外，还需要另个模块依赖：junit-jupiter-params，其主要就是提供了编写参数化测试 API。@ParameterizedTest 作为参数化测试的必要注解，替代了 @Test 注解。任何一个参数化测试方法都需要标记上该注解。

（1）基本数据源测试： @ValueSource

@ValueSource 是 JUnit 5 提供的最简单的数据参数源，支持 Java 的八大基本类型、字符串和 Class，使用时赋值给注解上对应类型属性，以数组方式传递。

（2）CSV 数据源测试：@CsvSource

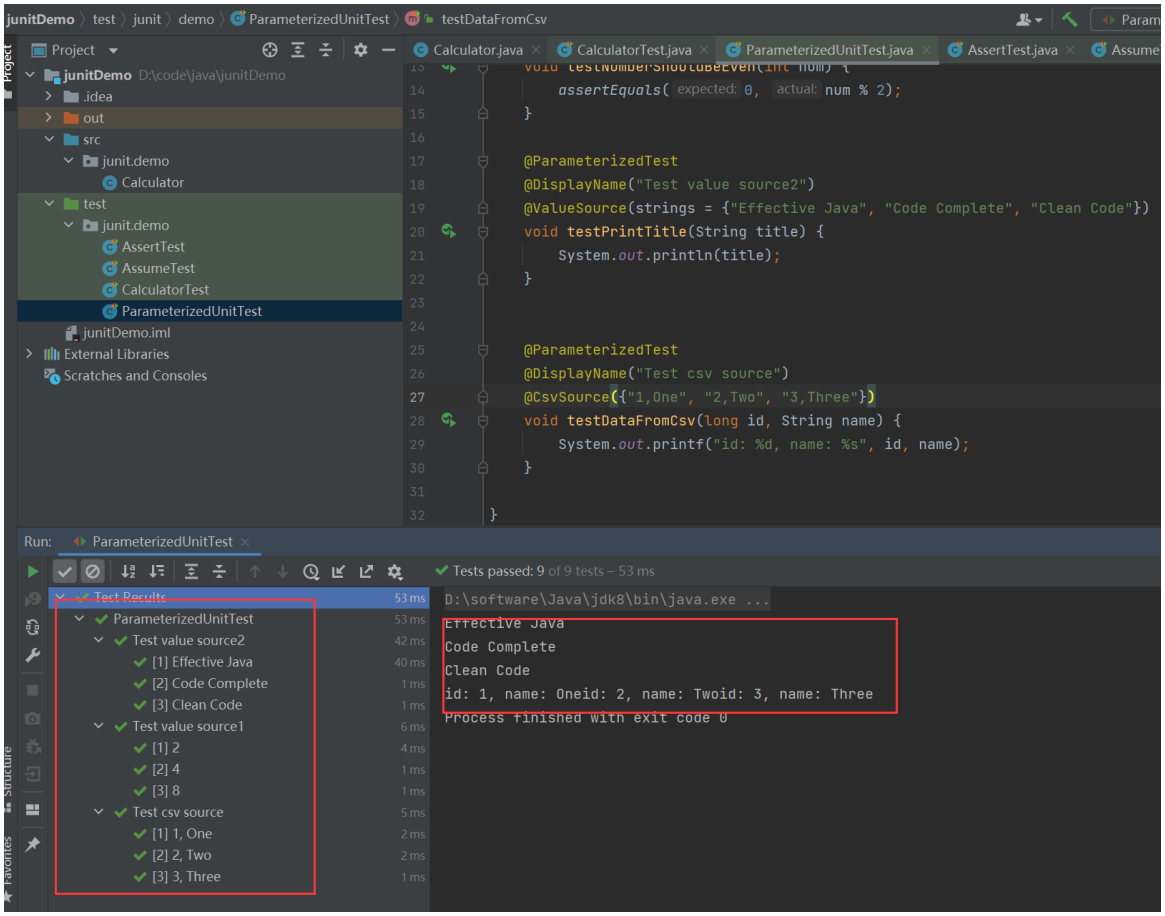
通过 @CsvSource 可以注入指定 CSV 格式 (comma-separated-values) 的一组数据，用每个逗号分隔的值来匹配一个测试方法对应的参数。

代码示例：

```
package junit.demo;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;
import static org.junit.jupiter.api.Assertions.*;

public class ParameterizedUnitTest {
        @ParameterizedTest
        @DisplayName("Test value source1")
        @ValueSource(ints = {2, 4, 8})
        void testNumberShouldBeEven(int num) {
```

```
        assertEquals(0, num % 2);
    }

    @ParameterizedTest
    @DisplayName("Test value source2")
    @ValueSource(strings = {"Effective Java", "Code Complete", "Clean Code"})
    void testPrintTitle(String title) {
        System.out.println(title);
    }


    @ParameterizedTest
    @DisplayName("Test csv source")
    @CsvSource({"1,One", "2,Two", "3,Three"})
    void testDataFromCsv(long id, String name) {
        System.out.printf("id: %d, name: %s", id, name);
    }

}
```

运行结果：



5. 本次迭代开发目标

本次迭代任务清单（Checklist）：

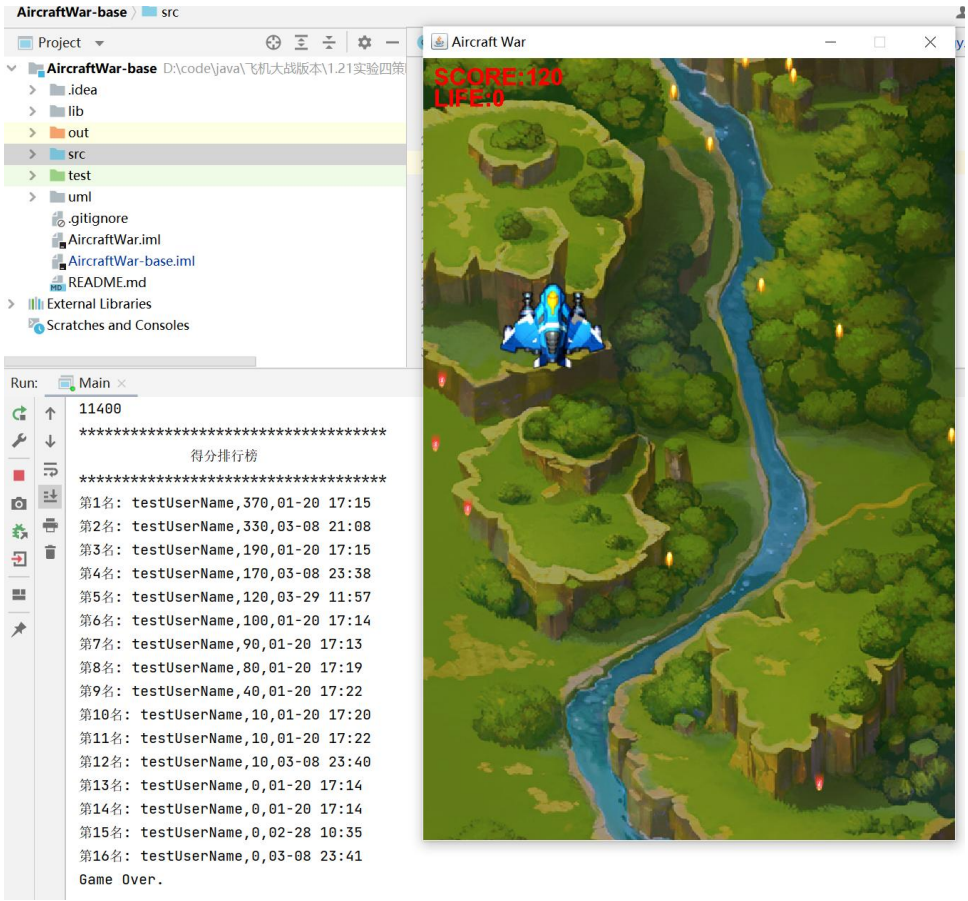
（1）**UML 类图绘制：**按照规范绘制数据访问对象（DAO）模式的 UML 结构类图

（2）**设计模式应用：**完成代码重构，采用 DAO 模式实现排行榜数据的持久化存储功能；

（3）**文件读写：**得分数据需存储在文件中，且每个游戏难度对应单独的存储文件，确保数据分离；

（4）**排行榜：**每局游戏结束后，将当局游戏得分、时间插入数据记录；并展示当前难度下的最新总分排行榜，榜单需包含名次、玩家名、得分及时间；

（5）**单元测试：**对英雄机类进行单元测试，至少选择 3 个方法（含其父类方法）作为测试对象，设计合理测试用例并完成测试。

注：本次实验仅需在游戏结束后记录该局分数，使用默认玩家名即可；排行榜无需实现界面展示及玩家交互功能，在控制台打印输出。

排行榜参考输出格式：